

Nafil Fadillah

20123958

3KB02

Pertemuan VII

```
In [1]: # To create tree
tree = {}
path = []
maxHeight, maxHeightNode = -1, -1

# Function to store the path from given vertex to the target vertex in a vector path

In [2]: def getDiameterPath(vertex, targetVertex, parent, path):

    # If the target node is found, push it into path vector
    if (vertex == targetVertex):
        path.append(vertex)
        return True

    for i in range(len(tree[vertex])):
        # To prevent visiting a node already visited
        if (tree[vertex][i] == parent):
            continue

        # Recursive call to the neighbours of current node inorder to get the path
        if (getDiameterPath(tree[vertex][i], targetVertex, vertex, path)):
            path.append(vertex)
            return True
    return False
```

Kode tersebut merupakan implementasi algoritma **Depth First Search (DFS)** pada struktur data **tree** yang direpresentasikan menggunakan **dictionary (adjacency list)**. Tujuan utamanya adalah untuk mencari **jalur (path) antara dua node**, yaitu dari node awal (vertex) ke node tujuan (targetVertex), sekaligus menyimpan jalur tersebut ke dalam list path.

Pada bagian awal, tree dibuat sebagai dictionary kosong {} yang nantinya akan berisi pasangan node dan daftar node yang terhubung dengannya. List path digunakan untuk menyimpan urutan node yang membentuk jalur yang ditemukan. Sementara itu, variabel maxHeight dan maxHeightNode disiapkan untuk kebutuhan lain seperti pencarian diameter tree, meskipun tidak langsung digunakan dalam fungsi ini.

Fungsi utama getDiameterPath(vertex, targetVertex, parent, path) bekerja secara rekursif. Fungsi ini memeriksa apakah node saat ini (vertex) adalah node tujuan. Jika ya, maka node tersebut dimasukkan ke dalam path dan fungsi mengembalikan nilai True sebagai tanda bahwa target telah ditemukan.

Jika belum, maka fungsi akan menelusuri semua tetangga dari node tersebut menggunakan perulangan pada tree[vertex]. Untuk setiap tetangga, dilakukan pengecekan apakah node tersebut adalah parent (untuk mencegah kembali ke node sebelumnya dan menghindari perulangan tak berujung). Jika bukan parent, fungsi akan dipanggil kembali secara rekursif pada node tetangga tersebut.

Apabila dari salah satu jalur rekursi ditemukan target, maka node saat ini akan dimasukkan ke dalam path saat proses backtracking (kembali dari rekursi). Dengan cara ini, path dibangun dari node tujuan kembali ke node awal.

Jika tidak ditemukan jalur ke target dari node tersebut, fungsi akan mengembalikan False.

Secara keseluruhan, kode ini bekerja dengan konsep **DFS + backtracking** untuk menemukan jalur antara dua node dalam tree tanpa kembali ke node yang sudah dikunjungi sebelumnya melalui parameter parent.

```
In [3]: # Function to obtain and return the farthest node from a given vertex
def farthestNode(vertex, parent, height):
    global maxHeight, maxHeightNode

    # If the current height is maximum so far, then save the current node
    if (height > maxHeight):
        maxHeight = height
        maxHeightNode = vertex

    # Iterate over all the neighbours of current node
    if (vertex in tree):
        for i in range(len(tree[vertex])):

            # This is to prevent visiting a already visited node
            if (tree[vertex][i] == parent):
                continue

            # Next call will be at 1 height higher than our current height
            farthestNode(tree[vertex][i], vertex, height + 1)
```

```
In [4]: # Function to add edges
def addedge(a, b):
    if (a not in tree):
        tree[a] = []

    tree[a].append(b)

    if (b not in tree):
        tree[b] = []

    tree[b].append(a)
```

Kode ini berisi dua fungsi utama dalam pengolahan tree (pohon) menggunakan Python, yaitu untuk membangun struktur tree dan mencari node terjauh (farthest node) dengan metode DFS (Depth First Search).

1. Fungsi farthestNode

Fungsi ini digunakan untuk mencari node yang paling jauh dari suatu node awal di dalam tree.

Prosesnya dimulai dari sebuah node (vertex) dan terus menjelajahi seluruh node yang terhubung secara rekursif. Setiap kali fungsi berpindah ke node lain, nilai jarak (height) akan bertambah satu.

Di dalam fungsi ini terdapat dua variabel global:

- maxHeight → menyimpan jarak terbesar yang ditemukan
- maxHeightNode → menyimpan node yang berada pada jarak terjauh tersebut

Jika jarak saat ini lebih besar dari maxHeight, maka nilai tersebut akan diperbarui.

Untuk menghindari perulangan ke node yang sama, fungsi ini menggunakan parameter parent agar tidak kembali ke node sebelumnya.

2. Fungsi addedge

Fungsi ini digunakan untuk membangun struktur tree dengan menambahkan hubungan (edge) antara dua node a dan b.

Jika node belum ada di dalam tree, maka akan dibuat terlebih dahulu sebagai list kosong. Setelah itu:

- node b dimasukkan ke daftar tetangga a
- node a dimasukkan ke daftar tetangga b

Hal ini membuat tree bersifat tidak berarah (undirected graph), karena hubungan berlaku dua arah.

Secara keseluruhan:

- addedge(a, b) → membentuk struktur tree dengan menghubungkan node
- farthestNode(...) → mencari node yang paling jauh dari suatu titik menggunakan DFS

Kode ini biasanya digunakan sebagai bagian dari algoritma untuk mencari diameter tree.

```
In [5]: def FindCenter(n):
# Now we will find the 1st farthest node from 0 (any arbitrary node)

# Perform DFS from 0 and update the maxHeightNode to obtain the farthest node from 0

# Reset to -1
maxHeight = -1

# Reset to -1
maxHeightNode = -1

farthestNode(0, -1, 0)

# Stores one end of the diameter
leaf1 = maxHeightNode

# Similarly the other end of the diameter
# Reset the maxHeight
maxHeight = -1
farthestNode(maxHeightNode, -1, 0)

# Stores the second end of the diameter
leaf2 = maxHeightNode

# Store the diameter into the vector path
path = []

# Diameter is equal to the path between the two farthest nodes leaf1 and leaf2
getDiameterPath(leaf1, leaf2, -1, path)

pathSize = len(path)

if (pathSize % 2 == 1):
    print(path[int((pathSize / 2)) * -1])
else:
    print(path[int((pathSize / 2))], ", ", path[int((pathSize - 1) / 2)], sep = "", end = "")

In [6]: N = 4

tree = {}
addege(1, 0)
addege(1, 2)
addege(1, 3)

FindCenter(N)

# This code is contributed by suresh07.

1
```

Kode tersebut merupakan implementasi untuk mencari center (titik tengah) dari sebuah tree dengan memanfaatkan konsep diameter tree dan algoritma Depth First Search (DFS).

Proses dimulai dengan mencari salah satu ujung diameter tree menggunakan fungsi `farthestNode` dari node awal (misalnya node 0). Hasilnya disimpan sebagai `leaf1`, yaitu node yang paling jauh dari titik awal tersebut. Setelah itu, pencarian dilakukan kembali dari `leaf1` untuk mendapatkan node terjauh lainnya yang disebut `leaf2`. Kedua node ini merupakan dua ujung dari diameter tree.

Selanjutnya, fungsi `getDiameterPath` digunakan untuk mencari jalur (path) yang menghubungkan `leaf1` dan `leaf2`. Jalur ini merepresentasikan diameter tree, yaitu lintasan terpanjang dalam tree.

Setelah jalur diameter diperoleh, panjang jalur dihitung menggunakan `pathSize`. Berdasarkan panjang ini, program menentukan titik tengah (center) tree. Jika jumlah node pada jalur ganjil, maka hanya ada satu node tengah yang menjadi center. Jika jumlahnya genap, maka terdapat dua node tengah yang keduanya dianggap sebagai center tree.

Pada contoh tree yang dibentuk dengan node 0, 1, 2, dan 3 yang terhubung melalui node 1 sebagai pusat, hasil akhirnya menunjukkan bahwa node 1 merupakan center dari tree tersebut.

Secara keseluruhan, kode ini bekerja dengan cara:

1. Mencari diameter tree menggunakan dua kali DFS
2. Mengambil jalur diameter
3. Menentukan titik tengah dari jalur tersebut sebagai center tree

Pertemuan VII

```
In [ ]: import sys

In [ ]: class Graph:
    # Constructor
    def __init__(self, edges, n):

        # A list of lists to represent an adjacency list
        self.adjList = [[] for _ in range(n)]

        # Add edges to the directed graph
        for (source, dest, weight) in edges:
            self.adjList[source].append((dest, weight))

        # Perform DFS on the graph and set the departure time of all vertices of the graph

In [ ]: def DFS(graph, v, discovered, departure, time):

    # Mark the current node as discovered
    discovered[v] = True

    # Set arrival time - not needed
    # time = time + 1

    # Do for every edge (v, u)
    for (u, w) in graph.adjList[v]:
        # if `u` is not yet discovered
        if not discovered[u]:
            time = DFS(graph, u, discovered, departure, time)

    # Ready to backtrack
    # Set departure time of vertex `v`
    departure[time] = v

    time = time + 1
    return time
```

Kode pada gambar tersebut menggambarkan bagaimana sebuah graf berarah direpresentasikan dan ditelusuri menggunakan algoritma Depth First Search (DFS) untuk menentukan urutan waktu selesai dari setiap simpul.

Pada bagian awal, program mengimpor modul `sys`, meskipun dalam potongan ini belum digunakan secara langsung. Selanjutnya dibuat sebuah kelas bernama `Graph` yang berfungsi sebagai struktur utama untuk menyimpan dan mengelola data graf. Ketika objek dari kelas ini dibuat, konstruktor (`__init__`) akan menerima dua parameter, yaitu daftar sisi (`edges`) dan jumlah simpul (`n`). Di dalam konstruktor tersebut, program membangun sebuah struktur data berupa adjacency list, yaitu daftar yang berisi list lain untuk merepresentasikan hubungan antar simpul. Setiap indeks pada adjacency list mewakili satu simpul, dan isi dari indeks tersebut adalah daftar simpul lain yang terhubung dengannya.

Kemudian, setiap edge yang diberikan akan diproses satu per satu. Untuk setiap pasangan source (asal), destination (tujuan), dan weight (bobot), program akan menambahkan tujuan dan bobot tersebut ke dalam adjacency list pada posisi simpul asal. Karena penambahan hanya dilakukan dari source ke destination, maka graf ini bersifat berarah.

Setelah struktur graf terbentuk, program mendefinisikan fungsi DFS (Depth First Search). Fungsi ini digunakan untuk menelusuri graf secara mendalam, yaitu dari satu simpul ke simpul lain hingga mencapai simpul terdalam sebelum kembali ke simpul sebelumnya. Fungsi DFS menerima beberapa parameter, yaitu graf itu sendiri, simpul awal yang akan dikunjungi, array penanda simpul yang sudah dikunjungi (`discovered`), array untuk menyimpan waktu selesai (`departure`), dan variabel waktu sebagai penghitung.

Ketika DFS dijalankan, langkah pertama yang dilakukan adalah menandai simpul saat ini sebagai sudah dikunjungi. Hal ini penting untuk menghindari pengulangan kunjungan pada simpul yang sama, terutama jika graf memiliki siklus. Setelah itu, program akan memeriksa semua simpul yang terhubung langsung dengan simpul saat ini melalui adjacency list. Jika ditemukan simpul yang belum dikunjungi, maka fungsi DFS akan dipanggil kembali secara rekursif untuk simpul tersebut. Proses ini akan terus berlanjut hingga mencapai simpul yang tidak memiliki tetangga yang belum dikunjungi.

Setelah semua simpul yang dapat dijangkau dari simpul tersebut telah dikunjungi, program akan melakukan proses backtracking, yaitu kembali ke simpul sebelumnya. Pada saat inilah waktu selesai (`departure time`) dari simpul dicatat ke dalam array `departure`. Nilai waktu kemudian ditambahkan sebagai penanda urutan selesai penelusuran.

Secara keseluruhan, kode ini tidak hanya melakukan traversal graf, tetapi juga mencatat urutan simpul berdasarkan waktu selesai penelusuran. Informasi ini sangat penting dalam berbagai aplikasi, seperti pengurutan topologis, analisis dependensi, dan pendeteksian struktur graf tanpa siklus (DAG).

```

In [ ]: # The function performs the topological sort on a given DAG and then finds
# the longest distance of all vertices from the given source by running one pass
# of the Bellman-Ford algorithm on edges of vertices in topological order
def findShortestDistance(graph, source, n):

    # `departure` stores the vertex number using departure time as an index
    departure = [-1] * n

    # To keep track of whether a vertex is discovered or not
    discovered = [False] * n
    time = 0

    # Perform DFS on all undiscovered vertices
    for i in range(n):
        if not discovered[i]:
            time = DFS(graph, i, discovered, departure, time)

    cost = [sys.maxsize] * n
    cost[source] = 0

    # Process the vertices in topological order, i.e., in order of their decreasing departure time in DFS
    for i in reversed(range(n)):

        # For each vertex in topological order, relax the cost of its adjacent vertices
        v = departure[i]

        # Edge from `v` to `u` having weight `w`
        for (u, w) in graph.adjList[v]:
            # if the distance to destination `u` can be shortened by
            # taking edge (v, u), update cost to the new lower value
            if cost[v] != sys.maxsize and cost[v] + w < cost[u]:
                cost[u] = cost[v] + w

    # Print shortest paths
    for i in range(n):
        print(f'dist({source}, {i}) = {cost[i]}')

In [1]: if __name__ == '__main__':

    # List of graph edges as per the above diagram
    edges = [
        (0, 6, 2), (1, 2, -4), (1, 4, 1), (1, 6, 8), (3, 0, 3), (3, 4, 5),
        (5, 1, 2), (7, 0, 6), (7, 1, -1), (7, 3, 4), (7, 5, -4)
    ]

    # Total number of nodes in the graph (Labelled from 0 to 7)
    n = 8

    # Build a graph from the given edges
    graph = Graph(edges, n)

    # Source vertex
    source = 7

    # Find the shortest distance of all vertices from the given source
    findShortestDistance(graph, source, n)

```

Kode dimulai dengan fungsi `findShortestDistance(graph, source, n)`. Fungsi ini bertujuan untuk menghitung jarak terpendek dari satu simpul sumber (`source`) ke semua simpul lain dalam graf.

Pertama, dibuat array `departure` dengan ukuran `n` dan diisi nilai `-1`. Array ini digunakan untuk menyimpan urutan simpul berdasarkan waktu selesai (`departure time`) yang dihasilkan dari DFS. Dengan kata lain, array ini nantinya akan merepresentasikan urutan topologis dari graf.

Selanjutnya, dibuat array `discovered` yang berisi nilai boolean (`True/False`) untuk menandai apakah suatu simpul sudah dikunjungi atau belum saat DFS dilakukan. Variabel `time` diinisialisasi dengan `0` sebagai penghitung waktu.

Kemudian program menjalankan DFS untuk semua simpul yang belum dikunjungi. Tujuannya adalah untuk memastikan seluruh graf ditelusuri, termasuk jika graf tidak terhubung penuh. Selama proses DFS ini, setiap simpul akan dicatat waktu selesainya ke dalam array `departure`. Hasil akhirnya adalah urutan simpul berdasarkan waktu selesai, yang bisa digunakan sebagai urutan topologis.

Setelah itu, program membuat array `cost` untuk menyimpan jarak dari simpul sumber ke semua simpul lainnya. Semua nilai awal diisi dengan nilai maksimum (`sys.maxsize`) sebagai

representasi tak hingga (∞), kecuali simpul sumber yang di-set bernilai 0 karena jarak dari sumber ke dirinya sendiri adalah nol.

Langkah berikutnya adalah bagian inti, yaitu memproses simpul dalam urutan topologis. Urutan ini diambil dari array departure. Karena DFS menyimpan simpul berdasarkan waktu selesai, maka dengan membaca array ini, kita mendapatkan urutan yang valid untuk memproses DAG.

Untuk setiap simpul v dalam urutan tersebut, program akan melihat semua tetangganya (u). Jika terdapat edge dari v ke u dengan bobot w , maka dilakukan proses relaksasi. Relaksasi adalah proses membandingkan jarak yang sudah ada dengan jarak baru yang mungkin lebih pendek melalui simpul v . Jika ditemukan jarak yang lebih kecil, maka nilai $cost[u]$ akan diperbarui.

Proses ini dilakukan satu kali untuk setiap simpul dalam urutan topologis, sehingga efisien dibandingkan algoritma lain seperti Dijkstra pada kasus DAG.

Terakhir, program mencetak hasil jarak terpendek dari simpul sumber ke semua simpul lainnya dalam format:

$dist(source, i) = \text{nilai}$

Pada bagian paling bawah (if `__name__ == '__main__':`), program mendefinisikan contoh graf dengan sejumlah edge yang memiliki bobot (termasuk bobot negatif, yang masih aman digunakan karena graf adalah DAG dan tidak memiliki siklus). Jumlah simpul ditentukan sebanyak 8 (dari 0 sampai 7), lalu graf dibangun menggunakan class Graph. Simpul sumber ditetapkan sebagai node 7. Setelah itu fungsi `findShortestDistance` dipanggil untuk menghitung dan menampilkan hasilnya.

Secara keseluruhan, kode ini menunjukkan cara yang efisien untuk mencari jarak terpendek pada graf berarah tanpa siklus dengan memanfaatkan DFS untuk mendapatkan urutan topologi, lalu melakukan relaksasi edge mengikuti urutan tersebut.